



ОНЛАЙН-ОБРАЗОВАНИЕ

**Проверить включена ли
запись?**



И в целом, ничего не забыто

Меня хорошо слышно && видно?



Напишите в чат, если есть проблемы!

Ставьте ☐ + если все хорошо

Ставьте ☐ - если есть проблемы

Основы архитектуры приложений



Поехали!

- ❖ Как пойдет занятие зависит от того, как будут задаваться вопросы
- ❖ Поэтому желательно вопросы задавать конце блока. Для этого у нас есть специальные слайды
- ❖ Но! Если что-то мешает дальнейшему пониманию темы лучше сразу задать вопрос в чат
- ❖ Оффтоп и выходящие за рамки занятия вопросы лучше задать в телеграм. Или в чат зума. Я их сохраню и отвечу после занятия
- ❖ В конце с вас отзыв)



- ❖ Поговорим про архитектуру в целом
- ❖ На примерах разберем несколько архитектурных принципов
- ❖ Подробно рассмотрим те “уровни” архитектуры, с которыми, в основном сталкиваются разработчики

01

Архитектура программного обеспечения



- Архитектура программного обеспечения — совокупность важнейших решений об организации программной системы ©
- Эти решения необходимо принять на разных уровнях: инфраструктура; компоненты системы; модули проекта; классы; методы.
- Уровень собственно системы один из самых ВЫСОКИХ
- Но архитектура метода+ не менее важна
- Плюс, разработчики чаще всего встречаются с последними двумя-тремя нижними уровнями

- В качестве примера выступает приложение для ведения заметок
- Оно в бесконечном цикле считывает пользовательский ввод ожидая выбора пункта меню
- Пользователь может создать новую заметку, редактировать или удалить старую, а так же вывести все заметки или ВЫЙТИ

Демо

(Знакомство с примером)



02

Архитектура уровня метода





- Это самый нижний уровень архитектуры
- Но даже на этом уровне архитектуру можно разделить на две части
 - Внешний вид (фасад)
 - Внутреннее устройство
- И начнем мы как раз с фасада)

- ~~Я пишу код и именую классы/поля/переменные так, чтобы мне было понятно~~
- Я пишу код и именую классы/поля/переменные так, чтобы было понятно другим



- Соблюдаются соглашения. Код форматируется в соответствии с принятым на проекте стандартом
- Названия классов, полей и переменных это существительные, а методы, действия
- Все названия несут смысловую нагрузку, отражают суть и не вводят в заблуждение
- Код читается сверху-вниз, слева-направо. Вызовы друг в друга не вкладываются.
- Вложенные блоки выносятся в методы

- Если вам захотелось перед тем или иным блоком кода написать комментарий, то этот код должен быть вынесен в метод
- В другой редакции этого правила код нужно выносить в метод если захотелось поставить пустую строку-разделитель. Но это очень жестко)
- Оба правила справедливы даже если речь идет о блоке из одной строки
- Такого рода подходы, кроме основной задачи разбиения на методы, позволяют писать самодокументированный код. Конечно при условии, что методам даются понятные, несущие смысловую нагрузку имена

- KISS (Keep it simple, stupid) – принцип, запрещающий использование более сложных средств, чем необходимо
- Разбивайте задачу на множество более маленьких задач
- Сохраняйте ваши методы маленькими. Каждый метод должен состоять не более чем из 30-40 строк
- Каждый метод должен решать одну маленькую задачу, а не множество
- Если в вашем методе множество условий, разбейте его на несколько

- DRY (Don't repeat yourself) – принцип разработки программного обеспечения, нацеленный на снижение повторения информации различного рода
- Т.е. похожий (совпадающий) код, который повторяется в разных местах можно привести к единому виду, вынести и переиспользовать
- Это не обязательно касается именно логики. Могут выноситься константы/сообщения и проч.
- Имеются противопоказания)

- YAGNI (You aren't gonna need it) – принцип проектирования ПО, при котором в качестве основного момента декларируется отказ от избыточной функциональности
- Т.е. отказ от добавления функциональности, в которой нет непосредственной надобности
- Такая функциональность, кроме того, что требует времени на реализацию еще и увеличивает сложность системы
- А так же, впоследствии, может выступать в качестве ограничения для ввода других функций т.к. ее существование всегда придется учитывать

Демо

(Архитектура уровня методов)



- Я написал всю программу в одном методе одного класса. Это было просто
 - + KISS
- Может мне стоит применить DRY или что-то еще? Или не стоит усложнять? Мне это может никогда не понадобится
 - + YAGNI (You aren't gonna need it)
- Я вынес строки в константы
 - + DRY
- Три из трех!

Вопросы?

(ставим “+” если вопросы есть и “-”
если вопросов нет)

03

Архитектура уровня классов





- На данном уровне мы оперируем уже не методами
- В качестве строительного материала мы уже используем классы и интерфейсы
- Тут на помощь приходят некоторые типовые архитектуры
- А так же новые (и старые) архитектурные принципы

- KISS, DRY, YAGNY
- SOLID – 5 основных принципов объектно-ориентированного программирования и проектирования:
 - Принцип единственной ответственности (single responsibility principle, SRP)
 - Принцип открытости/закрытости (open-closed principle, OCP)
 - Принцип подстановки Лисков (Liskov substitution principle, LSP)
 - Принцип разделения интерфейса (interface segregation principle, ISP)
 - Принцип инверсии зависимостей (dependency inversion principle, DIP)
- Принципы SOLID сформулированы для классов, но подходят, как для более нижних, так и для более верхних уровней проектирования

- Принцип единственной ответственности (single responsibility principle, SRP)
- Для каждого класса должно быть определено единственное назначение
- Все ресурсы, необходимые для его осуществления, должны быть инкапсулированы в этот класс и подчинены только этой задаче



Демо (Принцип единственной ответственности)



Вопросы?

(ставим “+” если вопросы есть и “-”
если вопросов нет)

- Принцип инверсии зависимостей (dependency inversion principle, DIP)
- Модули верхних уровней не должны зависеть от модулей нижних уровней
- Оба типа модулей должны зависеть от абстракций
- Абстракции не должны зависеть от деталей
- Детали должны зависеть от абстракций



Демо (

Принцип инверсии

зависимостей

)



Вопросы?

(ставим “+” если вопросы есть и “-”
если вопросов нет)

- Принцип открытости/закрытости (open-closed principle, OCP)
- Программные сущности должны быть открыты для расширения
- Но закрыты для изменения



- Принцип разделения интерфейса (interface segregation principle, ISP)
- Программные сущности не должны зависеть от методов, которые они не используют



Демо (

Принцип

открытости/закрытости

+

принцип разделения

интерфейса

)



Вопросы?

(ставим “+” если вопросы есть и “-”
если вопросов нет)

04

Выводы и советы



- Архитектурные принципы очень важны
- Но надо помнить, что они могут как помочь, так и навредить
- Особенно если трактовать их по своему
- В частности KISS и YAGNI иногда служат оправданием “грязного” кода
- А вообще архитектурные принципы это всего лишь инструменты
- Не религия и не истина в последней инстанции

- Тот или иной принцип можно применить в разной степени (не до конца)
- А между несколькими скорее всего придется искать компромисс
- Ну и не стоит к ним привязываться, скорее всего они с вами не навсегда
- Или со временем будут трактоваться по-другому, либо им придут на смену другие принципы
- А может несколько идеологий будут существовать одновременно и будут все правильные

Вопросы?

(ставим “+” если вопросы есть и “-”
если вопросов нет)

**Пожалуйста, пройдите
опрос**

Спасибо за внимание!

